Why Your Qubit Doesn't Give Exactly 70%/30%

Lab Handout

Contents

1 The Problem **2**

2 Why? Random Sampling + Binomial Distribution **2**
2.1 What is "Binomial"? 2
2.2 Exact Formula 2
2.3 Expected Value and Wobble (Standard Deviation) 2

3 Bell Curve Approximation (Central Limit Theorem) **2**
3.1 Variance of $\hat{p}$ 2

4 Normal Approximation and Confidence Intervals **3**

5 Why 1.96? The 95% Confidence Interval **3**
5.1 Step-by-Step Derivation 3
5.2 Common Confidence Levels 3

6 95% vs. 99% Confidence Intervals **3**

7 Confidence Intervals by Shots **4**

8 Key Takeaways **4**

9 Homework **4**

1 The Problem

Your code:

```
shots = 1024
outcomes = np.random.choice([0, 1], size=shots, p=[0.7, 0.3])
```

Expected: 70.0% $|0\rangle$, 30.0% $|1\rangle$
You get: 71.2% $|0\rangle$, 28.8% $|1\rangle$ (example)

2 Why? Random Sampling + Binomial Distribution

2.1 What is "Binomial"?

Bi-nomial = *two names*. Each trial has **2 outcomes**:

- Success = $|0\rangle$ ($p = 0.7$)
- Failure = $|1\rangle$ ($p = 0.3$)

Let X = number of $|0\rangle$ outcomes in $n = 1024$ shots:

$$\mathbf{X} \sim \text{Binomial}(\mathbf{n = 1024}, \mathbf{p = 0.7})$$

2.2 Exact Formula

$$P(X = k) = \binom{1024}{k} \cdot 0.7^k \cdot 0.3^{1024-k}$$

k = number of successes ($|0\rangle$ outcomes).
Examples:

- $P(X = 700)$ = probability of exactly 700 $|0\rangle$.
- $P(X = 717)$ = probability of exactly 717 $|0\rangle$ (near peak).
- $P(X = 900)$ = probability of exactly 900 $|0\rangle$ (very unlikely).

2.3 Expected Value and Wobble (Standard Deviation)

$$\mu = np = 1024 \times 0.7 = \mathbf{716.8}$$
$$\sigma = \sqrt{np(1-p)} = \sqrt{1024 \times 0.7 \times 0.3} = \mathbf{14.7}$$

Typical range: $717 \pm 15 = \mathbf{702 - 732}$ heads (68.6%-71.4%).

3 Bell Curve Approximation (Central Limit Theorem)

For large n, Binomial$(n, p) \approx$ Normal$(\mu = np, \sigma = \sqrt{np(1-p)})$.
Your measured probability $\hat{p} = X/n$:

$$\hat{p} \approx \text{Normal}\Big(0.70, \frac{\sqrt{0.7(1-0.7)}}{\sqrt{1024}}\Big)$$

3.1 Variance of $\hat{p}$

$$\text{Var}(\hat{p}) = \frac{p(1-p)}{n} = \frac{0.7 \times 0.3}{1024} = 0.000205$$
$$\sigma_{\hat{p}} = \sqrt{\frac{p(1-p)}{n}} = 0.0143$$

4 Normal Approximation and Confidence Intervals

Final form:

$$\hat{p} \approx \text{Normal}\Big(0.70,\, 0.0143\Big)$$

95% confidence interval:

$$\hat{p} \pm 1.96\sigma_{\hat{p}} = 0.70 \pm 0.028 = 67.2\% \text{ to } 72.8\%$$

Shots (n)	$\sigma_{\hat{p}}$	95% Margin	Typical Range
100	0.046	±9.0%	61–79%
1,000	0.014	±2.8%	67–73%
10,000	**0.0046**	**±0.9%**	**69.1–70.9%**
100,000	0.0014	±0.3%	69.7–70.3%

Table 1: Precision vs. shots (from $\sigma_{\hat{p}} = \sqrt{p(1-p)/n}$)

5 Why 1.96? The 95% Confidence Interval

5.1 Step-by-Step Derivation

We want 95% confidence ⇒ 2.5% in each tail:

1. Total tail area = 5% = 0.05
2. Each tail = 0.025
3. Find z where right tail = 0.025
4. From tables: $z_{0.975} = 1.96$

5.2 Common Confidence Levels

90%	±1.645	(5% each tail)
95%	**±1.96**	**(2.5% each tail)**
99%	±2.576	(0.5% each tail)

6 95% vs. 99% Confidence Intervals

For your experiment ($n = 1024, p = 0.7$):

$$\sigma_{\hat{p}} = 0.0143$$

Confidence	Formula	Margin	Range	Shots Needed
95%	$\pm 1.96 \times 0.0143$	±2.8%	67.2–72.8%	1,000
99%	$\pm 2.576 \times 0.0143$	**±3.7%**	**66.3–73.7%**	**1,700**

Table 2: 95% vs 99% confidence for the experiment

99% confidence means a wider range because we demand higher certainty.

Shots	95% Margin	99% Margin	95% Range	99% Range
100	±9.0%	±11.8%	61–79%	58–82%
1,000	±2.8%	±3.7%	67–73%	66–74%
10,000	±0.9%	±1.2%	69.1–70.9%	68.8–71.2%
100,000	±0.3%	±0.4%	69.7–70.3%	69.6–70.4%

Table 3: Precision vs. confidence level

7 Confidence Intervals by Shots

8 Key Takeaways

1. Your qubit is perfect; your **measurement shows statistical noise**.
2. 1024 shots $\Rightarrow$ ±3% error (normal).
3. 10k shots $\Rightarrow$ publication quality (±1%).
4. Binomial$(n, p) \to$ Normal for $n \geq 30$.

9 Homework

Perform a statistical analysis of the following experiment:

```python
import numpy as np

print("Simulating shot experiments with precision analysis\n")

for i in range(5):
    shots = 3000
    outcomes = np.random.choice([0, 1], size=shots, p=[0.6, 0.4])

    # Measure both proportions (they always sum to 1)
    p1_measured = np.sum(outcomes == 1) / shots
    p0_measured = 1 - p1_measured
    print(f"Exp #{i+1}: {p0_measured:.1%} |0> and {p1_measured:.1%} |1>")
    print(f"Number of |0>: {np.sum(outcomes ==0)}")
    print(f"Number of |1>: {np.sum(outcomes ==1)}")
```

Submit by the start of M2